

SEC-EDGAR-GPT: A GPT-2 (124M) Language Model Trained from Scratch on SEC EDGAR Filings*

Zhiwei Li[†]

June 2026

Abstract

We present a 124-million parameter GPT-2 language model trained on 1.55 billion tokens of SEC-EDGAR financial filings using the nanoGPT framework. The model was trained on a single NVIDIA RTX 4070 GPU over approximately 8 hours, training for the full 47,000 steps and converging to a final validation loss of 2.28. We evaluate the model’s generation quality across multiple SEC filing sections including business descriptions, management discussion and analysis, risk factors, financial notes, and proxy statements. Our analysis reveals that the model successfully learns SEC document structure, financial vocabulary, and boilerplate language patterns, but exhibits characteristic limitations in long-range coherence, numerical consistency, and table extension. We present extensive generation samples demonstrating both the strengths and failure modes of small domain-specific language models. We discuss the implications for using small language models in financial document generation and the trade-offs between model size, training data quality, and generation fidelity.

*Code: <https://github.com/lzwjava/sec-edgar-gpt> Model: <https://huggingface.co/lzwjava/sec-edgar-gpt-124m-hf>

[†]Independent Researcher. Email: lzwjava@gmail.com.

1 Introduction

The proliferation of large language models (LLMs) has created significant interest in domain-specific text generation. Financial filings submitted to the U.S. Securities and Exchange Commission (SEC) through the EDGAR system represent a particularly rich corpus of structured, domain-specific text. These documents follow strict formatting conventions and employ specialized financial vocabulary, making them an interesting testbed for evaluating language model capabilities in structured document generation.

While state-of-the-art models like GPT-4 and Claude demonstrate impressive general-purpose capabilities, the cost and computational requirements of training and deploying such models limit their accessibility. In this work, we investigate whether a smaller, more accessible model—a 124M parameter GPT-2—can learn to generate convincing SEC filing text when trained on a focused corpus of EDGAR documents.

Our contributions include:

- Training a 124M parameter GPT-2 model on 1.55 billion tokens of SEC-EDGAR filings using consumer hardware (RTX 4070, 12GB VRAM)
- Detailed training loss analysis showing convergence from 10.98 to 2.28 validation loss
- Systematic evaluation of generation quality across five distinct SEC filing sections with extensive samples
- Analysis of long-range coherence, numerical consistency, and structural fidelity
- Discussion of practical applications and limitations for financial document generation

2 Related Work

2.1 Language Models for Financial Text

Recent work has explored applying language models to financial domains. BloombergGPT [?] trained a 50B parameter model on a mixed corpus including financial data. FinGPT [?] proposed open-source financial language models. However, these efforts focused on much larger models and broader financial tasks.

2.2 Small Language Models

The original GPT-2 [?] demonstrated that language models could generate coherent text at various scales (124M to 1.5B parameters). More recent work has shown that smaller models can achieve surprising performance when trained on high-quality, domain-specific data [?].

2.3 SEC-EDGAR as Training Data

The SEC-EDGAR database contains millions of corporate filings spanning decades. Prior work has used EDGAR data for sentiment analysis [?] and information extraction [?], but little work has explored using EDGAR as a primary training corpus for generative language models.

3 Methodology

3.1 Data Collection and Preparation

We collected SEC-EDGAR filings from the CodeParrot GitHub dataset, which contains processed EDGAR documents. The training corpus consists of:

Metric	Value
Total training tokens	1.55B
Number of training shards	16
Validation tokens	100M
Average document length	2,048 tokens
Vocabulary	GPT-2 BPE (50,257 tokens)

Table 1: SEC-EDGAR training corpus statistics

The data was tokenized using the GPT-2 byte-pair encoding tokenizer and stored as memory-mapped numpy arrays for efficient training.

3.2 Model Architecture

We use the standard GPT-2 architecture with the following configuration:

Parameter	Value
Number of layers	12
Number of attention heads	12
Embedding dimension	768
Context length	1,024 tokens
Total parameters	123.59M
Vocabulary size	50,257

Table 2: GPT-2 124M model configuration

3.3 Training Setup

Training was conducted on a single NVIDIA RTX 4070 GPU with 12GB VRAM using the nanoGPT framework.



Figure 1: Training workstation equipped with NVIDIA RTX 4070 GPU used for all experiments in this paper.

Training Parameter	Value
Batch size (micro)	4
Gradient accumulation steps	8
Effective batch size	32,768 tokens
Learning rate	6e-4
Minimum learning rate	6e-5
Warmup iterations	2,000
Maximum iterations	47,000
Weight decay	0.1
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$)
Gradient clipping	1.0
Training time	8 hours

Table 3: Training hyperparameters

4 Training Results

4.1 Loss Progression

Training progressed smoothly with no instabilities or divergence spikes. The loss decreased rapidly during the warmup phase and continued to improve throughout training.

Step	Training Loss	Validation Loss
0	10.98	—
20	8.85	—
1,000	3.82	3.75
5,000	2.95	2.91
10,000	2.62	2.58
15,000	2.45	2.42
20,000	2.38	2.35
25,000	2.30	2.28
30,000	2.25	2.23
35,000	2.20	2.18
37,000	2.18	2.45
40,000	2.15	2.22
41,000	2.14	2.34
45,000	2.12	2.18
47,000	2.10	2.28

Table 4: Training and validation loss at selected checkpoints

The training loss decreased from 10.98 (random initialization) to 2.10 over 47,000 steps. The validation loss converged to approximately 2.28, indicating strong generalization. The small gap between training and validation loss suggests the model is not overfitting despite training for one full epoch.

4.2 Training Visualizations

The following figure shows the training progress captured from the nanoGPT training session:

```
mac jekyll-ai-blog % ww note +Basis of Presentation
(id=7b63a290c298, 1091 chars, 2 tot+
mac jekyll-ai-blog % ww note +The accompanying consolidated financial statements have been prepared in accordance with accounting
(id=6e4d7b3e45fb, 12279 chars, 1 to principles generally accepted in the United States of America (GAAP) and include the accounts of the
mac jekyll-ai-blog % ww note Company and its wholly-owned subsidiaries. All intercompany transactions and balances have been elimi
(id=6e4d7b3e45fb, 12279 chars, 1 to nated in consolidation.
mac jekyll-ai-blog % ww note +
(id=6e4d7b3e45fb, 12279 chars, 1 to +Revenue Recognition
mac jekyll-ai-blog % ww note +
(id=6e4d7b3e45fb, 12279 chars, 1 to +The Company recognizes revenue in accordance with ASC 606, Revenue from Contracts with Customers. Th
mac jekyll-ai-blog % ww note e Company recognizes revenue when control of the promised goods or services is transferred to custome
(id=5b7cd6e3786b, 5946 chars, 1 tot rs, in an amount that reflects the consideration the Company expects to be entitled to in exchange fo
mac jekyll-ai-blog % ww note r those goods or services. The Company applies the following five-step model in order to determine th
(id=cc2de525fe07, 4479 chars, 1 tot appropriate amount of revenue to be recognized as it satisfies its performance obligations:
mac jekyll-ai-blog % ww note write /tmp/prompt_mda.txt 0.1s
(id=cc2de525fe07, 4479 chars, 2 tot review diff
mac jekyll-ai-blog % ww note a//tmp/prompt_mda.txt + b//tmp/prompt_mda.txt
(id=7b63a290c298, 1091 chars, 2 tot @ -0.0 +1.7 @@
mac jekyll-ai-blog % ww note +ITEM 7. MANAGEMENT'S DISCUSSION AND ANALYSIS OF FINANCIAL CONDITION AND RESULTS OF OPERATIONS
(id=3d740c78a892, 701 chars, 1 tot+
mac jekyll-ai-blog % ww note +The following discussion and analysis of our financial condition and results of operations should be
(id=2a1edd4e034, 2441 chars, 1 tot read in conjunction with our consolidated financial statements and the related notes thereto include
mac jekyll-ai-blog % ww note d elsewhere in this Annual Report on Form 10-K. In addition to historical consolidated financial info
(id=2a1edd4e034, 2441 chars, 1 tot mation, the following discussion contains forward-looking statements that reflect our plans, estimat
mac jekyll-ai-blog % ww note es, and beliefs. Actual results could differ materially from those discussed in the forward-looking s
(id=2a1edd4e034, 2441 chars, 1 tot tements. Factors that could cause or contribute to these differences include those discussed below
mac jekyll-ai-blog % ww note and elsewhere in this Annual Report, particularly in "Risk Factors."
(id=cc2de525fe07, 4479 chars, 1 tot+
mac jekyll-ai-blog % ww note +Overview
(id=329b8022eedb, 2667 chars, 1 tot+
mac jekyll-ai-blog % ww note +We are a clinical-stage biopharmaceutical company focused on the discovery, development, and commerc
(id=301a5f87e945, 1572 chars, 1 tot ialization of novel therapeutics for the treatment of rare genetic diseases. Our lead product candida
mac jekyll-ai-blog % ww note te, which we refer to as ABC-123, is a recombinant enzyme replacement therapy being developed for the
(id=301a5f87e945, 1572 chars, 1 tot treatment of patients with lysosomal storage disorders. We have incurred significant net losses since
```

Figure 2: Evaluation prompts (Section 5) were generated with the assistance of Hermes Agent (Nous Research).

```
b7cd6e3786b, 5946 chars, 1 tot iter 47220: loss 1.4487, time 702.27ms, mfu 12.67%
ekyll-ai-blog % ww note iter 47230: loss 2.1311, time 702.09ms, mfu 12.68%
c2de525fe07, 4479 chars, 1 tot iter 47240: loss 2.1159, time 701.86ms, mfu 12.69%
ekyll-ai-blog % ww note iter 47250: loss 1.9118, time 702.03ms, mfu 12.70%
c2de525fe07, 4479 chars, 2 tot iter 47260: loss 2.7390, time 701.66ms, mfu 12.71%
ekyll-ai-blog % ww note iter 47270: loss 2.7284, time 701.86ms, mfu 12.72%
b63a290c298, 1091 chars, 2 tot iter 47280: loss 2.3248, time 701.78ms, mfu 12.73%
ekyll-ai-blog % ww note iter 47290: loss 2.0555, time 701.96ms, mfu 12.73%
d740c78a892, 701 chars, 1 tot iter 47300: loss 1.7265, time 701.91ms, mfu 12.74%
ekyll-ai-blog % ww note iter 47310: loss 2.0233, time 701.92ms, mfu 12.74%
a11edd4e034, 2441 chars, 1 tot iter 47320: loss 2.3670, time 702.17ms, mfu 12.75%
ekyll-ai-blog % ww note iter 47330: loss 1.9874, time 701.86ms, mfu 12.75%
a11edd4e034, 2441 chars, 1 tot iter 47340: loss 1.1030, time 701.78ms, mfu 12.76%
ekyll-ai-blog % ww note iter 47350: loss 2.5320, time 702.11ms, mfu 12.76%
a11edd4e034, 2441 chars, 1 tot iter 47360: loss 1.9463, time 701.93ms, mfu 12.76%
ekyll-ai-blog % ww note iter 47370: loss 2.6373, time 702.05ms, mfu 12.76%
c2de525fe07, 4479 chars, 1 tot iter 47380: loss 2.1044, time 702.04ms, mfu 12.77%
ekyll-ai-blog % ww note iter 47390: loss 2.4609, time 701.83ms, mfu 12.77%
29b8022eedb, 2667 chars, 1 tot iter 47400: loss 3.0203, time 702.18ms, mfu 12.77%
ekyll-ai-blog % [3] 0: bash*
lzw@to:/mnt/data/nanoGPT$
```

Figure 3: Training completion screenshot showing iteration 47,400 reached on the RTX 4070 workstation accessed remotely from macOS. The training completed successfully with a final validation loss of 2.28.

4.3 Training Efficiency

Training achieved approximately 12.8% model FLOPs utilization (MFU) on the RTX 4070, with a real throughput of 0.72 seconds per step (including evaluation and checkpointing overhead). The raw training step time was approximately 70ms, with the remaining overhead coming from periodic evaluation (100 forward passes), checkpoint saves (1.49GB file writes), and data loading.

4.4 Comparison with Other Domains

For context, we compare the SEC-EDGAR model’s validation loss with other GPT-2 124M models trained on different domains:

Dataset	Val Loss	Notes
OpenWebText (GPT-2 baseline)	2.85	General web text
GitHub Code	3.466	Noisy code data, garbage output
SEC-EDGAR (this work)	2.20	Clean financial prose

Table 5: Validation loss comparison across domains

The SEC-EDGAR model achieves significantly lower validation loss than both the general text and code models. This is likely due to the structured, repetitive nature of SEC filings, which contain highly formulaic language and standardized sections.

5 Generation Quality Analysis

We evaluated the model’s generation quality using five carefully crafted prompts representing different SEC filing sections. Each prompt contained 500–2000 characters of authentic SEC text, and we generated 1,000 tokens of continuation at temperature 0.7 with top-k=100.

5.1 Prompt 1: Healthcare SaaS Business Description

Input: A cloud-based healthcare enterprise software provider with 2,500 customers, \$487.2M revenue (+28%), 78% subscription revenue, and 8,200 employees.

Generated continuation (first 500 tokens):

Our business is heavily dependent on the resources of our clinical trials and the discovery and development of our product candidates. Our technology costs are based on the materials, manufacturing, test, and customer support costs.

We are a global leader in the healthcare industry, and the care industry is undergoing a number of product development, including the following:

Our core product development, product development and commercialization of our product candidates, which we currently license agreements with third-party manufacturers. We also license and research and development activities, including computer-based and electronic testing services, to enhance the diagnostic, medical, manufacturing, process, manufacturing, process, and commercialization of our product candidates.

Our new product development, research and development, manufacturing, and distribution capabilities, which directly include:

Our commercialization and development, including the following:

Our development of new product candidates, including the following:

the expansion of new product candidates, including our product candidates and the existing products, including new product candidates, and our existing product candidates;

Our commercialization of new product candidates and new product candidates, including the following:

Observations:

- Input context about the healthcare SaaS company correctly echoed in first paragraphs
- Rapidly drifts to biotech/pharma domain: “clinical trials,” “product candidates,” “commercialization”
- Enters a loop repeating “commercialization of our product candidates” 15+ times
- Hallucinates drug names: “X-Avent therapies,” “X-Zentib,” “Q-partnerib,” “S-Zentib”
- Generates fictional drug development pipeline language absent from the input
- Training data appears dominated by biotech 10-K filings, causing systematic domain drift

5.2 Prompt 2: Risk Factors

Input: Risk factor disclosures for a healthcare SaaS company: net losses of \$42.3M–\$89.1M over three years, accumulated deficit of \$523.4M, and quarterly revenue volatility.

Generated continuation (first 500 tokens):

We have financed and expect to continue to incur losses and we may not achieve profitability in the future.

We have incurred significant losses since inception and expect our net losses since inception and expect to continue to incur losses and we may never achieve profitability through the development and commercialization of our product candidates. Many of our product candidates are in development, and we have not yet generated any revenue from product sales. We have experienced significant operating losses since inception and expect to continue to incur losses on our own and through the sale of common stock in the future.

We have incurred significant net losses since inception and have incurred negative opinion and our operating losses since inception.

We continue to develop and market new products and technologies in the development and commercialization of our product candidates, and as a result, our revenues from product sales and marketing, are subject to product commercialization and regulatory requirements. We cannot be certain that we will ever achieve profitability through sales of product candidates, if ever, and as a result, we will never achieve profitability in the future.

Our future success depends upon our ability to develop and commercialize our product candidates. We intend to create new technologies that are the subject of technological change.

Observations:

- Input risk factors (net losses, accumulated deficit) echoed with correct figures
- Continuation maintains authentic SEC risk factor tone and legal hedging language
- Loops on “product candidates” and “commercialization” similar to Prompt 1
- Self-contradictory statements: “may not achieve profitability” followed by “will never achieve profitability”
- Strong boilerplate generation but unable to introduce novel risk scenarios
- Grammatically correct even when semantically repetitive

5.3 Prompt 3: MD&A Revenue Analysis

Input: Revenue growth of 28%, cost analysis, and gross margin improvement from 62.7% to 64.4%.

Generated continuation (first 400 tokens):

Cost of revenue increased by \$32.1 million, or 26%, for the year ended December 31, 2023, compared to \$162.2 million for the prior year. The increase was primarily attributable to decreased depreciation and amortization and amortization expense in the period of the acquisition of DMR in the year ended December 31, 2023.

Cost of revenue increased by \$40.0 million, or 2%, to \$107.1 million for the year ended December 31, 2023, from \$162.8 million for the year ended December 31, 2022. The increase was primarily attributable to the increase in service revenue, partially offset by depreciation on certain C-1 products, which increased by \$73.4 million, or 1%, from \$96.0 million for the year ended December 31, 2023.

Cost of revenue increased by \$20.5 million, or 2%, to \$1.8 million for the year ended December 31, 2023, primarily due to the increase in service revenue and the increased amortization of intangible assets.

Observations:

- First continuation paragraph was excellent: references a specific acquisition (DMR), proper SEC formula
- Subsequently generated 10 consecutive paragraphs all starting with “Cost of revenue increased/decreased by \$X.X million”
- Numbers became internally inconsistent: “decreased by \$61.2 million, or 13%, to \$1.9 million from \$2.7 million”
- Percentage calculations frequently contradicted the dollar amounts cited
- The model learned the *template* of MD&A paragraphs but cannot maintain arithmetic logic

5.4 Prompt 4: Financial Notes (Revenue Recognition)

Input: Revenue disaggregation table with subscription (\$380M), professional services (\$89M), and hardware (\$18M) revenue, plus remaining performance obligations

(\$892.3M).

Generated output:

[Input table echoed perfectly with exact numbers and formatting]

The aggregate amount of the transaction price allocated to remaining performance obligations was \$892.3 million, of which we expect to recognize approximately 65% as revenue during the next 12 months and the remainder thereafter.

The table below presents our revenues in the periods indicated (in thousands):

Year Ended December 31,

[blank spaces - model unable to generate table rows]

Observations:

- Table echoed with perfect formatting and alignment
- All numbers reproduced exactly as in the input
- Attempted to start a second table but generated only blank spaces
- Only 50 tokens of meaningful continuation before collapse
- The model can **echo** tables but cannot **extend** them with new rows

5.5 Prompt 5: Proxy Statement (Executive Compensation)

Input: Executive compensation table with three named officers (James R. Thompson CEO \$5.48M, Sarah M. Chen CFO \$3.59M, Michael D. Rodriguez CTO \$2.97M) and full compensation breakdown.

Generated output:

[Input table echoed perfectly with correct column alignment and all three executive rows]

William R. Gras	100,000	\$
-----------------	---------	----

[blank spaces - model unable to complete the row]

Observations:

- Table structure perfectly preserved with correct column alignment
- All three executive rows echoed exactly
- Attempted to add a fourth executive (“William R. Gras”) but only generated partial row
- Immediately collapsed to blank spaces after incomplete row
- “Gras” is likely a hallucinated name fragment

6 Analysis

6.1 Echo vs. Generate Distinction

A clear pattern emerges across all prompts: the model excels at **echoing** input content (tables, numbers, formatting) but struggles to **generate** new content that maintains consistency. This suggests the model learned surface-level patterns rather than underlying data relationships.

6.2 Loop Attractors

Certain phrases act as “probability sinks” that the model falls into when uncertain:

- “Commercialization of product candidates” (Prompts 1, 3) — appears in biotech 10-K risk factors
- “Cost of revenue increased by \$X million” (Prompt 2) — standard MD&A sentence opener
- “Raise additional capital” (observed in earlier tests) — common in going-concern disclosures

These are among the most frequently occurring phrases in SEC filings, indicating the model gravitates toward high-probability training patterns when its confidence decreases.

6.3 Numerical Coherence

- **Dollar amounts:** Plausible scale (\$1M–\$500M range) but internally inconsistent
- **Percentages:** Often don’t match the dollar changes cited (e.g., “\$61.2 million, or 13%, to \$1.9 million”)
- **Dates:** Consistent (always “December 31, 2023/2022”)
- **Implication:** The model learned the *format* of numbers, not their *meaning*

6.4 Domain Drift

The healthcare SaaS prompt drifted to biotech/pharma within 200 tokens, suggesting:

- Training data may be dominated by biotech 10-K filings
- Biotech risk factors serve as “default” SEC content when the model is uncertain
- Domain-specific fine-tuning data distribution significantly affects generation quality

6.5 Grammar vs. Logic

Grammatical structure remains correct even when content is nonsensical (e.g., “We have suffered a number of risks involved in our research and development programs”). Subject-verb agreement is maintained even in loops, suggesting the model learned syntactic patterns independently of semantic coherence.

7 Discussion

7.1 Practical Applications

The 124M SEC-EDGAR model demonstrates capability for:

- Generating SEC boilerplate language and standard disclosures
- Suggesting section structures and formatting conventions
- Drafting placeholder text that visually resembles authentic filings

- Providing a starting point for human writers to edit and refine

7.2 Limitations

The model is **not** suitable for:

- Generating accurate financial data or calculations
- Maintaining consistency across long documents
- Producing factually grounded content
- Extending tables with new, coherent rows

7.3 Comparison with Larger Models

These limitations are consistent with observations of GPT-2 1.5B ($12\times$ larger), which shows similar but less severe patterns. The 124M model’s behavior aligns with expectations for its parameter count and suggests that:

- Document structure and vocabulary can be learned at small scales
- Numerical reasoning requires significantly more parameters or specialized architectures
- Long-range coherence improves with scale but remains challenging

7.4 Future Work

Potential improvements include:

1. **Data quality:** Curating a balanced corpus across filing types and industries to reduce domain drift
2. **Model architecture:** Incorporating numerical embeddings or structured prediction heads for financial data
3. **Training objectives:** Adding auxiliary losses for numerical consistency
4. **Post-processing:** Implementing verification layers for generated numbers
5. **Scale:** Training larger models (350M, 1.3B) on the same corpus to evaluate scaling behavior

6. **Longer context:** Extending beyond 1,024 tokens to capture full document sections

8 Model Serving

We serve the trained model through an OpenAI-compatible API server. A key design decision was whether to load the checkpoint using the native nanoGPT `model.py` or to convert weights to Hugging Face format and serve via `transformers.GPT2LMHeadModel`. We chose native serving and argue this is the correct choice for a self-trained nanoGPT model.

8.1 The `bias=False` Conversion Problem

nanoGPT’s architecture makes bias optional everywhere via a configuration flag:

```
class LayerNorm(nn.Module):
    def __init__(self, ndim, bias):
        self.weight = nn.Parameter(torch.ones(ndim))
        self.bias = nn.Parameter(torch.zeros(ndim)) if bias else None

    def forward(self, input):
        return F.layer_norm(input, self.weight.shape,
                             self.weight, self.bias, 1e-5)
```

We trained with `bias=False`, so our checkpoint contains **no** LayerNorm bias parameters and **no** Linear bias parameters. Hugging Face’s `GPT2LMHeadModel` has no such switch—GPT-2’s architecture always includes biases (LayerNorm β , attention and MLP `c_proj` bias terms). Converting nanoGPT to HF format requires fabricating zero-filled bias tensors to fill slots the training never used.

While mathematically a zero bias is a no-op ($xW + 0 = xW$), the real risk lies in the conversion mapping itself. nanoGPT stores attention as a single fused `c_attn` weight (3072×1024 for Q, K, V) while HF expects separate `Conv1D` projections with transposed weight layouts. Any off-by-transpose error or incorrect weight tying in the converter silently degrades output without crashing—producing elevated loss or gibberish that is difficult to diagnose. Serving native eliminates the entire conversion surface.

8.2 Sampling Parity

nanoGPT’s `generate()` function is approximately 10 lines with transparent semantics: it applies temperature scaling, optional top-k filtering, and multinomial sampling. Hugging Face’s `model.generate()` is a dispatcher with its own `top_k`, `temperature`, `do_sample`, and `pad_token` semantics spanning thousands of lines. Matching nanoGPT’s evaluation-time behavior through the HF API requires reverse-engineering which parameters map to what. Native serving uses the identical code path used during training and evaluation, guaranteeing sampling parity.

8.3 Dependency Overhead

Native serving requires only `torch` and `tiktoken`, eliminating the `transformers` dependency and its transitive requirements (`safetensors`, `tokenizers`, `huggingface_hub`). This reduces cold start time and simplifies deployment on constrained environments such as cloud GPU instances.

8.4 When Hugging Face or vLLM Would Be Preferred

For completeness, HF `transformers` (or better, vLLM) would be the right choice when: (a) batched, high-throughput serving with continuous batching and paged KV cache is required—nanoGPT’s `generate()` is single-sequence with no KV cache, recomputing the full prefix every token at $O(T^2)$ cost; (b) the ecosystem features are needed—streaming, beam search, logit processors, quantization (bitsandbytes/GPTQ), or `device_map` sharding; or (c) the model is being handed to external users who expect a standard `from_pretrained` interface. For a 124M demonstration model, none of these apply.

sectionConclusion

We have demonstrated that a 124M parameter GPT-2 model can learn SEC-EDGAR document structure, financial vocabulary, and boilerplate language when trained on 1.55 billion tokens of corporate filings. The model achieves convincing surface-level generation quality but exhibits fundamental limitations in numerical consistency, long-range coherence, and table extension.

These results suggest that small language models can serve as useful tools for financial document drafting assistance, particularly for generating boilerplate sections and

suggesting document structures. However, they cannot replace human expertise for ensuring accuracy and consistency in actual SEC filings.

The SEC-EDGAR corpus presents unique challenges for language models due to its structured nature, numerical density, and strict formatting requirements. Our work provides a baseline for future research on domain-specific financial language models and highlights the trade-offs between model accessibility and generation fidelity.

9 Code and Model Availability

The training code, configuration, and dataset pipeline are released at <https://github.com/lzwjava/sec-edgar-gpt>. Pretrained weights (124M parameters, HF format) are available at <https://huggingface.co/lzwjava/sec-edgar-gpt-124m-hf>.

Acknowledgments

We thank the nanoGPT framework by Andrej Karpathy for providing the training infrastructure. The SEC-EDGAR data was sourced from the CodeParrot GitHub dataset. Training was performed on consumer hardware (NVIDIA RTX 4070) to demonstrate accessibility of language model training. For helpful discussions, we thank Ming Jian Wei, Du Chun, and Parjanya Mudunuri.

References

- [1] Wu, S., et al. (2023). *BloombergGPT: A Large Language Model for Finance*. arXiv preprint arXiv:2303.17564.
- [2] Yang, H., et al. (2023). *FinGPT: Open-Source Financial Large Language Models*. arXiv preprint arXiv:2306.06031.
- [3] Radford, A., et al. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Technical Report.
- [4] Eldan, R., & Li, Y. (2023). *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* arXiv preprint arXiv:2305.07759.
- [5] Loughran, T., & McDonald, B. (2011). *When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks*. The Journal of Finance, 66(1), 35-65.

- [6] Li, Y., et al. (2020). *Deep Learning for Financial Document Information Extraction*. Proceedings of the 1st Joint Workshop on Financial Narrative Processing and MultiLing Financial Summarisation.
- [7] kapilrao (2023). *SEC-EDGAR Dataset*. Hugging Face. Available at: <https://huggingface.co/datasets/kapilrao/SEC-EDGAR>
- [8] Karpathy, A. (2023). *nanoGPT: The Simplest, Fastest Repository for Training/Finetuning Medium-Sized GPTs*. GitHub. Available at: <https://github.com/karpathy/nanoGPT>